

To initialize the face tracking operation, create a new **FaceTracker** object by calling **CreateAsync**. Initialize the desired timer interval and then create the timer. The **ProcessCurrentVideoFrame** helper method will be called every time the specified interval elapses.

C#

```
this.faceTracker = await FaceTracker.CreateAsync();
TimeSpan timerInterval = TimeSpan.FromMilliseconds(66); // 15 fps
this.frameProcessingTimer =
    Windows.System.Threading.ThreadPoolTimer.CreatePeriodicTimer(new
        Windows.System.Threading.TimerElapsedHandler(ProcessCurrentVideoFrame),
        timerInterval);
```

The **ProcessCurrentVideoFrame** helper is called asynchronously by the timer, so the method first calls the semaphore's **Wait** method to see if a tracking operation is ongoing, and if it is the method returns without trying to detect faces. At the end of this method, the semaphore's **Release** method is called, which allows the subsequent call to **ProcessCurrentVideoFrame** to continue.

The **FaceTracker** class operates on **VideoFrame** objects. There are multiple ways you can obtain a **VideoFrame** including capturing a preview frame from a running **MediaCapture** object or by implementing the **ProcessFrame** method of the **IBasicVideoEffect**. This example uses an undefined helper method that returns a video frame, **GetLatestFrame**, as a placeholder for this operation. For information about getting video frames from the preview stream of a running media capture device, see [Get a preview frame](#).

As with **FaceDetector**, the **FaceTracker** supports a limited set of pixel formats. This example abandons face detection if the supplied frame is not in the Nv12 format.

Call **ProcessNextFrameAsync** to retrieve a list of **DetectedFace** objects representing the faces in the frame. After you have the list of faces, you can display them in the same manner described above for face detection. Note that, because the face tracking helper method is not called on the UI thread, you must make any UI updates in within a call **CoreDispatcher.RunAsync**.

C#

```
public async void ProcessCurrentVideoFrame(ThreadPoolTimer timer)
{
    if (!frameProcessingSemaphore.Wait(0))
    {
        return;
    }

    VideoFrame currentFrame = await GetLatestFrame();
```

```

        // Use FaceDetector.GetSupportedBitmapPixelFormat and
        IsBitmapPixelFormatSupported to dynamically
        // determine supported formats
        const BitmapPixelFormat faceDetectionPixelFormat =
        BitmapPixelFormat.Nv12;

        if (currentFrame.SoftwareBitmap.BitmapPixelFormat !=
        faceDetectionPixelFormat)
        {
            return;
        }

        try
        {
            IList<DetectedFace> detectedFaces = await
            faceTracker.ProcessNextFrameAsync(currentFrame);

            var previewFrameSize = new
            Windows.Foundation.Size(currentFrame.SoftwareBitmap.PixelWidth,
            currentFrame.SoftwareBitmap.PixelHeight);
            var ignored =
            this.Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, ()
            =>
            {
                this.SetupVisualization(previewFrameSize, detectedFaces);
            });
        }
        catch (Exception e)
        {
            // Face tracking failed
        }
        finally
        {
            frameProcessingSemaphore.Release();
        }

        currentFrame.Dispose();
    }

```

Related topics

- [Scene analysis for media capture](#)
- [Basic Face Detection sample](#) ↗
- [Basic Face Tracking sample](#) ↗
- [Camera](#)
- [Basic photo, video, and audio capture with MediaCapture](#)
- [Media playback](#)

Custom video effects

Article • 10/20/2022

This article describes how to create a Windows Runtime component that implements the [IBasicVideoEffect](#) interface to create custom effects for video streams. Custom effects can be used with several different Windows Runtime APIs including [MediaCapture](#), which provides access to a device's camera, and [MediaComposition](#), which allows you to create complex compositions out of media clips.

Add a custom effect to your app

A custom video effect is defined in a class that implements the [IBasicVideoEffect](#) interface. This class can't be included directly in your app's project. Instead, you must use a Windows Runtime component to host your video effect class.

Add a Windows Runtime component for your video effect

1. In Microsoft Visual Studio, with your solution open, go to the **File** menu and select **Add->New Project**.
2. Select the **Windows Runtime Component (Universal Windows)** project type.
3. For this example, name the project *VideoEffectComponent*. This name will be referenced in code later.
4. Click **OK**.
5. The project template creates a class called *Class1.cs*. In **Solution Explorer**, right-click the icon for *Class1.cs* and select **Rename**.
6. Rename the file to *ExampleVideoEffect.cs*. Visual Studio will show a prompt asking if you want to update all references to the new name. Click **Yes**.
7. Open *ExampleVideoEffect.cs* and update the class definition to implement the [IBasicVideoEffect](#) interface.

C#

```
public sealed class ExampleVideoEffect : IBasicVideoEffect
```

You need to include the following namespaces in your effect class file in order to access all of the types used in the examples in this article.

C#

```
using Windows.Media.Effects;  
using Windows.Media.MediaProperties;  
using Windows.Foundation.Collections;
```

```
using Windows.Graphics.DirectX.Direct3D11;
using Windows.Graphics.Imaging;
```

Implement the IBasicVideoEffect interface using software processing

Your video effect must implement all of the methods and properties of the [IBasicVideoEffect](#) interface. This section walks you through a simple implementation of this interface that uses software processing.

Close method

The system will call the [Close](#) method on your class when the effect should shut down. You should use this method to dispose of any resources you have created. The argument to the method is a [MediaEffectClosedReason](#) that lets you know whether the effect was closed normally, if an error occurred, or if the effect does not support the required encoding format.

C#

```
public void Close(MediaEffectClosedReason reason)
{
    // Dispose of effect resources
}
```

DiscardQueuedFrames method

The [DiscardQueuedFrames](#) method is called when your effect should reset. A typical scenario for this is if your effect stores previously processed frames to use in processing the current frame. When this method is called, you should dispose of the set of previous frames you saved. This method can be used to reset any state related to previous frames, not only accumulated video frames.

C#

```
private int frameCount;
public void DiscardQueuedFrames()
{
    frameCount = 0;
}
```

IsReadOnly property

The [IsReadOnly](#) property lets the system know if your effect will write to the output of the effect. If your app does not modify the video frames (for example, an effect that only performs analysis of the video frames), you should set this property to true, which will cause the system to efficiently copy the frame input to the frame output for you.

Tip

When the [IsReadOnly](#) property is set to true, the system copies the input frame to the output frame before [ProcessFrame](#) is called. Setting the [IsReadOnly](#) property to true does not restrict you from writing to the effect's output frames in [ProcessFrame](#).

C#

```
public bool IsReadOnly { get { return false; } }
```

SetEncodingProperties method

The system calls [SetEncodingProperties](#) on your effect to let you know the encoding properties for the video stream upon which the effect is operating. This method also provides a reference to the Direct3D device used for hardware rendering. The usage of this device is shown in the hardware processing example later in this article.

C#

```
private VideoEncodingProperties encodingProperties;  
public void SetEncodingProperties(VideoEncodingProperties  
encodingProperties, IDirect3DDevice device)  
{  
    this.encodingProperties = encodingProperties;  
}
```

SupportedEncodingProperties property

The system checks the [SupportedEncodingProperties](#) property to determine which encoding properties are supported by your effect. Note that if the consumer of your effect can't encode video using the properties you specify, it will call [Close](#) on your effect and will remove your effect from the video pipeline.

C#

```
public IReadOnlyList<VideoEncodingProperties> SupportedEncodingProperties
{
    get
    {
        var encodingProperties = new VideoEncodingProperties();
        encodingProperties.Subtype = "ARGB32";
        return new List<VideoEncodingProperties>() { encodingProperties };

        // If the list is empty, the encoding type will be ARGB32.
        // return new List<VideoEncodingProperties>();
    }
}
```

ⓘ Note

If you return an empty list of **VideoEncodingProperties** objects from **SupportedEncodingProperties**, the system will default to ARGB32 encoding.

SupportedMemoryTypes property

The system checks the **SupportedMemoryTypes** property to determine whether your effect will access video frames in software memory or in hardware (GPU) memory. If you return **MediaMemoryTypes.Cpu**, your effect will be passed input and output frames that contain image data in **SoftwareBitmap** objects. If you return **MediaMemoryTypes.Gpu**, your effect will be passed input and output frames that contain image data in **IDirect3DSurface** objects.

C#

```
public MediaMemoryTypes SupportedMemoryTypes { get { return
MediaMemoryTypes.Cpu; } }
```

ⓘ Note

If you specify **MediaMemoryTypes.GpuAndCpu**, the system will use either GPU or system memory, whichever is more efficient for the pipeline. When using this value, you must check in the **ProcessFrame** method to see whether the **SoftwareBitmap** or **IDirect3DSurface** passed into the method contains data, and then process the frame accordingly.

TimeIndependent property

The **TimeIndependent** property lets the system know if your effect does not require uniform timing. When set to true, the system can use optimizations that enhance effect performance.

C#

```
public bool TimeIndependent { get { return true; } }
```

SetProperties method

The **SetProperties** method allows the app that is using your effect to adjust effect parameters. Properties are passed as an **IPropertySet** map of property names and values.

C#

```
private IPropertySet configuration;
public void SetProperties(IPropertySet configuration)
{
    this.configuration = configuration;
}
```

This simple example will dim the pixels in each video frame according to a specified value. A property is declared and TryGetValue is used to get the value set by the calling app. If no value was set, a default value of .5 is used.

C#

```
public double FadeValue
{
    get
    {
        object val;
        if (configuration != null && configuration.TryGetValue("FadeValue",
out val))
        {
            return (double)val;
        }
        return .5;
    }
}
```

ProcessFrame method

The **ProcessFrame** method is where your effect modifies the image data of the video. The method is called once per frame and is passed a **ProcessVideoFrameContext** object. This object contains an input **VideoFrame** object that contains the incoming frame to be processed and an output **VideoFrame** object to which you write image data that will be passed on to rest of the video pipeline. Each of these **VideoFrame** objects has a **SoftwareBitmap** property and a **Direct3DSurface** property, but which of these can be used is determined by the value you returned from the **SupportedMemoryTypes** property.

This example shows a simple implementation of the **ProcessFrame** method using software processing. For more information about working with **SoftwareBitmap** objects, see [Imaging](#). An example **ProcessFrame** implementation using hardware processing is shown later in this article.

Accessing the data buffer of a **SoftwareBitmap** requires COM interop, so you should include the **System.Runtime.InteropServices** namespace in your effect class file.

C#

```
using System.Runtime.InteropServices;
```

Add the following code inside the namespace for your effect to import the interface for accessing the image buffer.

C#

```
[ComImport]
[Guid("5B0D3235-4DBA-4D44-865E-8F1D0E4FD04D")]
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
unsafe interface IMemoryBufferByteAccess
{
    void GetBuffer(out byte* buffer, out uint capacity);
}
```

ⓘ Note

Because this technique accesses a native, unmanaged image buffer, you will need to configure your project to allow unsafe code.

1. In Solution Explorer, right-click the VideoEffectComponent project and select **Properties**.

2. Select the **Build** tab.
3. Select the **Allow unsafe code** check box.

Now you can add the **ProcessFrame** method implementation. First, this method obtains a **BitmapBuffer** object from both the input and output software bitmaps. Note that the output frame is opened for writing and the input for reading. Next, an **IMemoryBufferReference** is obtained for each buffer by calling **CreateReference**. Then, the actual data buffer is obtained by casting the **IMemoryBufferReference** objects as the COM interop interface defined above, **IMemoryByteAccess**, and then calling **GetBuffer**.

Now that the data buffers have been obtained, you can read from the input buffer and write to the output buffer. The layout of the buffer is obtained by calling **GetPlaneDescription**, which provides information on the width, stride, and initial offset of the buffer. The bits per pixel is determined by the encoding properties set previously with the **SetEncodingProperties** method. The buffer format information is used to find the index into the buffer for each pixel. The pixel value from the source buffer is copied into the target buffer, with the color values being multiplied by the **FadeValue** property defined for this effect to dim them by the specified amount.

C#

```
public unsafe void ProcessFrame(ProcessVideoFrameContext context)
{
    using (BitmapBuffer buffer =
context.InputFrame.SoftwareBitmap.LockBuffer(BitmapBufferAccessMode.Read))
        using (BitmapBuffer targetBuffer =
context.OutputFrame.SoftwareBitmap.LockBuffer(BitmapBufferAccessMode.Write))
        {
            using (var reference = buffer.CreateReference())
            using (var targetReference = targetBuffer.CreateReference())
            {
                byte* dataInBytes;
                uint capacity;
                ((IMemoryBufferByteAccess)reference).GetBuffer(out dataInBytes,
out capacity);

                byte* targetDataInBytes;
                uint targetCapacity;
                ((IMemoryBufferByteAccess)targetReference).GetBuffer(out
targetDataInBytes, out targetCapacity);

                var fadeValue = FadeValue;

                // Fill-in the BGRA plane
```

```

        BitmapPlaneDescription bufferLayout =
buffer.GetPlaneDescription(0);
        for (int i = 0; i < bufferLayout.Height; i++)
        {
            for (int j = 0; j < bufferLayout.Width; j++)
            {
                byte value = (byte)((float)j / bufferLayout.Width *
255);

                int bytesPerPixel = 4;
                if (encodingProperties.Subtype != "ARGB32")
                {
                    // If you support other encodings, adjust index into
the buffer accordingly
                }

                int idx = bufferLayout.StartIndex + bufferLayout.Stride
* i + bytesPerPixel * j;

                targetDataInBytes[idx + 0] = (byte)(fadeValue *
(float)dataInBytes[idx + 0]);
                targetDataInBytes[idx + 1] = (byte)(fadeValue *
(float)dataInBytes[idx + 1]);
                targetDataInBytes[idx + 2] = (byte)(fadeValue *
(float)dataInBytes[idx + 2]);
                targetDataInBytes[idx + 3] = dataInBytes[idx + 3];
            }
        }
    }
}

```

Implement the IBasicVideoEffect interface using hardware processing

Creating a custom video effect by using hardware (GPU) processing is almost identical to using software processing as described above. This section will show the few differences in an effect that uses hardware processing. This example uses the Win2D Windows Runtime API. For more information about using Win2D, see the [Win2D documentation](#).

Use the following steps to add the Win2D NuGet package to the project you created as described in the **Add a custom effect to your app** section at the beginning of this article.

To add the Win2D NuGet package to your effect project

1. In **Solution Explorer**, right-click the **VideoEffectComponent** project and select **Manage NuGet Packages**.
2. At the top of the window, select the **Browse** tab.
3. In the search box, enter **Win2D**.
4. Select **Win2D.uwp**, and then select **Install** in the right pane.
5. The **Review Changes** dialog shows you the package to be installed. Click **OK**.
6. Accept the package license.

In addition to the namespaces included in the basic project setup, you will need to include the following namespaces provided by Win2D.

C#

```
using Microsoft.Graphics.Canvas.Effects;
using Microsoft.Graphics.Canvas;
```

Because this effect will use GPU memory for operating on the image data, you should return **MediaMemoryTypes.Gpu** from the **SupportedMemoryTypes** property.

C#

```
public MediaMemoryTypes SupportedMemoryTypes { get { return
MediaMemoryTypes.Gpu; } }
```

Set the encoding properties that your effect will support with the **SupportedEncodingProperties** property. When working with Win2D, you must use ARGB32 encoding.

C#

```
public IReadOnlyList<VideoEncodingProperties> SupportedEncodingProperties {
    get
    {
        var encodingProperties = new VideoEncodingProperties();
        encodingProperties.Subtype = "ARGB32";
        return new List<VideoEncodingProperties>() { encodingProperties };
    }
}
```

Use the **SetEncodingProperties** method to create a new Win2D **CanvasDevice** object from the **IDirect3DDevice** passed into the method.

C#

```
private CanvasDevice canvasDevice;
public void SetEncodingProperties(VideoEncodingProperties
encodingProperties, IDirect3DDevice device)
{
    canvasDevice = CanvasDevice.CreateFromDirect3D11Device(device);
}
```

The **SetProperties** implementation is identical to the previous software processing example. This example uses a **BlurAmount** property to configure a Win2D blur effect.

C#

```
private IPropertySet configuration;
public void SetProperties(IPropertySet configuration)
{
    this.configuration = configuration;
}
```

C#

```
public double BlurAmount
{
    get
    {
        object val;
        if (configuration != null && configuration.TryGetValue("BlurAmount",
out val))
        {
            return (double)val;
        }
        return 3;
    }
}
```

The last step is to implement the **ProcessFrame** method that actually processes the image data.

Using Win2D APIs, a **CanvasBitmap** is created from the input frame's **Direct3DSurface** property. A **CanvasRenderTarget** is created from the output frame's **Direct3DSurface** and a **CanvasDrawingSession** is created from this render target. A new Win2D **GaussianBlurEffect** is initialized, using the **BlurAmount** property our effect exposes via **SetProperties**. Finally, the **CanvasDrawingSession.DrawImage** method is called to draw the input bitmap to the render target using the blur effect.

C#

```

public void ProcessFrame(ProcessVideoFrameContext context)
{
    using (CanvasBitmap inputBitmap =
        CanvasBitmap.CreateFromDirect3D11Surface(canvasDevice,
            context.InputFrame.Direct3DSurface))
        using (CanvasRenderTarget renderTarget =
            CanvasRenderTarget.CreateFromDirect3D11Surface(canvasDevice,
                context.OutputFrame.Direct3DSurface))
            using (CanvasDrawingSession ds = renderTarget.CreateDrawingSession())
            {
                var gaussianBlurEffect = new GaussianBlurEffect
                {
                    Source = inputBitmap,
                    BlurAmount = (float)BlurAmount,
                    Optimization = EffectOptimization.Speed
                };

                ds.DrawImage(gaussianBlurEffect);
            }
}

```

Adding your custom effect to your app

To use your video effect from your app, you must add a reference to the effect project to your app.

1. In Solution Explorer, under your app project, right-click **References** and select **Add reference**.
2. Expand the **Projects** tab, select **Solution**, and then select the check box for your effect project name. For this example, the name is *VideoEffectComponent*.
3. Click **OK**.

Add your custom effect to a camera video stream

You can set up a simple preview stream from the camera by following the steps in the article [Simple camera preview access](#). Following those steps will provide you with an initialized **MediaCapture** object that is used to access the camera's video stream.

To add your custom video effect to a camera stream, first create a new **VideoEffectDefinition** object, passing in the namespace and class name for your effect. Next, call the **MediaCapture** object's **AddVideoEffect** method to add your effect to the specified stream. This example uses the **MediaStreamType.VideoPreview** value to

specify that the effect should be added to the preview stream. If your app supports video capture, you could also use **MediaStreamType.VideoRecord** to add the effect to the capture stream. **AddVideoEffect** returns an **IMediaExtension** object representing your custom effect. You can use the **SetProperties** method to set the configuration for your effect.

After the effect has been added, **StartPreviewAsync** is called to start the preview stream.

C#

```
var videoEffectDefinition = new
VideoEffectDefinition("VideoEffectComponent.ExampleVideoEffect");

IMediaExtension videoEffect =
    await mediaCapture.AddVideoEffectAsync(videoEffectDefinition,
MediaStreamType.VideoPreview);

videoEffect.SetProperties(new PropertySet() { { "FadeValue", .25 } });

await mediaCapture.StartPreviewAsync();
```

Add your custom effect to a clip in a MediaComposition

For general guidance for creating media compositions from video clips, see [Media compositions and editing](#). The following code snippet shows the creation of a simple media composition that uses a custom video effect. A **MediaClip** object is created by calling **CreateFromFileAsync**, passing in a video file that was selected by the user with a **FileOpenPicker**, and the clip is added to a new **MediaComposition**. Next a new **VideoEffectDefinition** object is created, passing in the namespace and class name for your effect to the constructor. Finally, the effect definition is added to the **VideoEffectDefinitions** collection of the **MediaClip** object.

C#

```
MediaComposition composition = new MediaComposition();
var clip = await MediaClip.CreateFromFileAsync(pickedFile);
composition.Clips.Add(clip);

var videoEffectDefinition = new
VideoEffectDefinition("VideoEffectComponent.ExampleVideoEffect", new
PropertySet() { { "FadeValue", .5 } });

clip.VideoEffectDefinitions.Add(videoEffectDefinition);
```

Related topics

- [Simple camera preview access](#)
- [Media compositions and editing](#)
- [Win2D documentation](#) 
- [Media playback](#)

Custom audio effects

Article • 10/20/2022

This article describes how to create a Windows Runtime component that implements the [IBasicAudioEffect](#) interface to create custom effects for audio streams. Custom effects can be used with several different Windows Runtime APIs including [MediaCapture](#), which provides access to a device's camera, [MediaComposition](#), which allows you to create complex compositions out of media clips, and [AudioGraph](#) which allows you to quickly assemble a graph of various audio input, output, and submix nodes.

Add a custom effect to your app

A custom audio effect is defined in a class that implements the [IBasicAudioEffect](#) interface. This class can't be included directly in your app's project. Instead, you must use a Windows Runtime component to host your audio effect class.

Add a Windows Runtime component for your audio effect

1. In Microsoft Visual Studio, with your solution open, go to the **File** menu and select **Add->New Project**.
2. Select the **Windows Runtime Component (Universal Windows)** project type.
3. For this example, name the project *AudioEffectComponent*. This name will be referenced in code later.
4. Click **OK**.
5. The project template creates a class called *Class1.cs*. In **Solution Explorer**, right-click the icon for *Class1.cs* and select **Rename**.
6. Rename the file to *ExampleAudioEffect.cs*. Visual Studio will show a prompt asking if you want to update all references to the new name. Click **Yes**.
7. Open **ExampleAudioEffect.cs** and update the class definition to implement the [IBasicAudioEffect](#) interface.

C#

```
public sealed class ExampleAudioEffect : IBasicAudioEffect
```

You need to include the following namespaces in your effect class file in order to access all of the types used in the examples in this article.

C#

```
using Windows.Media.Effects;
using Windows.Media.MediaProperties;
using Windows.Foundation.Collections;
using System.Runtime.InteropServices;
using Windows.Media;
using Windows.Foundation;
```

Implement the `IBasicAudioEffect` interface

Your audio effect must implement all of the methods and properties of the `IBasicAudioEffect` interface. This section walks you through a simple implementation of this interface to create a basic echo effect.

SupportedEncodingProperties property

The system checks the `SupportedEncodingProperties` property to determine which encoding properties are supported by your effect. Note that if the consumer of your effect can't encode audio using the properties you specify, the system will call `Close` on your effect and will remove your effect from the audio pipeline. In this example, `AudioEncodingProperties` objects are created and added to the returned list to support 44.1 kHz and 48 kHz, 32-bit float, mono encoding.

C#

```
public IReadOnlyList<AudioEncodingProperties> SupportedEncodingProperties
{
    get
    {
        var supportedEncodingProperties = new List<AudioEncodingProperties>
        ();
        AudioEncodingProperties encodingProps1 =
        AudioEncodingProperties.CreatePcm(44100, 1, 32);
        encodingProps1.Subtype = MediaEncodingSubtypes.Float;
        AudioEncodingProperties encodingProps2 =
        AudioEncodingProperties.CreatePcm(48000, 1, 32);
        encodingProps2.Subtype = MediaEncodingSubtypes.Float;

        supportedEncodingProperties.Add(encodingProps1);
        supportedEncodingProperties.Add(encodingProps2);

        return supportedEncodingProperties;
    }
}
```

SetEncodingProperties method

The system calls [SetEncodingProperties](#) on your effect to let you know the encoding properties for the audio stream upon which the effect is operating. In order to implement an echo effect, this example uses a buffer to store one second of audio data. This method provides the opportunity to initialize the size of the buffer to the number of samples in one second of audio, based on the sample rate in which the audio is encoded. The delay effect also uses an integer counter to keep track of the current position in the delay buffer. Since **SetEncodingProperties** is called whenever the effect is added to the audio pipeline, this is a good time to initialize that value to 0. You may also want to capture the **AudioEncodingProperties** object passed into this method to use elsewhere in your effect.

C#

```
private float[] echoBuffer;
private int currentActiveSampleIndex;
private AudioEncodingProperties currentEncodingProperties;
```

C#

```
public void SetEncodingProperties(AudioEncodingProperties
encodingProperties)
{
    currentEncodingProperties = encodingProperties;
    echoBuffer = new float[encodingProperties.SampleRate]; // exactly one
second delay
    currentActiveSampleIndex = 0;
}
```

SetProperties method

The [SetProperties](#) method allows the app that is using your effect to adjust effect parameters. Properties are passed as an [IPropertySet](#) map of property names and values.

C#

```
IPropertySet configuration;
public void SetProperties(IPropertySet configuration)
{
    this.configuration = configuration;
}
```

This simple example will mix the current audio sample with a value from the delay buffer according the value of the **Mix** property. A property is declared and TryGetValue is used to get the value set by the calling app. If no value was set, a default value of .5 is used. Note that this property is read-only. The property value must be set using **SetProperties**.

C#

```
public float Mix
{
    get
    {
        object val;
        if (configuration != null && configuration.TryGetValue("Mix", out
val))
        {
            return (float)val;
        }
        return .5f;
    }
}
```

ProcessFrame method

The **ProcessFrame** method is where your effect modifies the audio data of the stream. The method is called once per frame and is passed a **ProcessAudioFrameContext** object. This object contains an input **AudioFrame** object that contains the incoming frame to be processed and an output **AudioFrame** object to which you write audio data that will be passed on to rest of the audio pipeline. An audio frame is a buffer of audio samples representing a short slice of audio data.

Accessing the data buffer of a **AudioFrame** requires COM interop, so you should include the **System.Runtime.InteropServices** namespace in your effect class file and then add the following code inside the namespace for your effect to import the interface for accessing the audio buffer.

C#

```
[ComImport]
[Guid("5B0D3235-4DBA-4D44-865E-8F1D0E4FD04D")]
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]
unsafe interface IMemoryBufferByteAccess
{
    void GetBuffer(out byte* buffer, out uint capacity);
}
```

ⓘ Note

Because this technique accesses a native, unmanaged image buffer, you will need to configure your project to allow unsafe code.

1. In Solution Explorer, right-click the AudioEffectComponent project and select **Properties**.
2. Select the **Build** tab.
3. Select the **Allow unsafe code** check box.

Now you can add the **ProcessFrame** method implementation to your effect. First, this method obtains a **AudioBuffer** object from both the input and output audio frames. Note that the output frame is opened for writing and the input for reading. Next, an **IMemoryBufferReference** is obtained for each buffer by calling **CreateReference**. Then, the actual data buffer is obtained by casting the **IMemoryBufferReference** objects as the COM interop interface defined above, **IMemoryByteAccess**, and then calling **GetBuffer**.

Now that the data buffers have been obtained, you can read from the input buffer and write to the output buffer. For each sample in the inputbuffer, the value is obtained and multiplied by $1 - \text{Mix}$ to set the dry signal value of the effect. Next, a sample is retrieved from the current position in the echo buffer and multiplied by **Mix** to set the wet value of the effect. The output sample is set to the sum of the dry and wet values. Finally, each input sample is stored in the echo buffer and the current sample index is incremented.

C#

```
unsafe public void ProcessFrame(ProcessAudioFrameContext context)
{
    AudioFrame inputFrame = context.InputFrame;
    AudioFrame outputFrame = context.OutputFrame;

    using (AudioBuffer inputBuffer =
inputFrame.LockBuffer(AudioBufferAccessMode.Read),
           outputBuffer =
outputFrame.LockBuffer(AudioBufferAccessMode.Write))
        using (IMemoryBufferReference inputReference =
inputBuffer.CreateReference(),
               outputReference =
outputBuffer.CreateReference())
        {
            byte* inputDataInBytes;
            byte* outputDataInBytes;
```

```

        uint inputCapacity;
        uint outputCapacity;

        ((IMemoryBufferByteAccess)inputReference).GetBuffer(out
inputDataInBytes, out inputCapacity);
        ((IMemoryBufferByteAccess)outputReference).GetBuffer(out
outputDataInBytes, out outputCapacity);

        float* inputDataInFloat = (float*)inputDataInBytes;
        float* outputDataInFloat = (float*)outputDataInBytes;

        float inputData;
        float echoData;

        // Process audio data
        int dataInFloatLength = (int)inputBuffer.Length / sizeof(float);

        for (int i = 0; i < dataInFloatLength; i++)
        {
            inputData = inputDataInFloat[i] * (1.0f - this.Mix);
            echoData = echoBuffer[currentActiveSampleIndex] * this.Mix;
            outputDataInFloat[i] = inputData + echoData;
            echoBuffer[currentActiveSampleIndex] = inputDataInFloat[i];
            currentActiveSampleIndex++;

            if (currentActiveSampleIndex == echoBuffer.Length)
            {
                // Wrap around (after one second of samples)
                currentActiveSampleIndex = 0;
            }
        }
    }
}

```

Close method

The system will call the **Close** method on your class when the effect should shut down. You should use this method to dispose of any resources you have created. The argument to the method is a **MediaEffectClosedReason** that lets you know whether the effect was closed normally, if an error occurred, or if the effect does not support the required encoding format.

C#

```

public void Close(MediaEffectClosedReason reason)
{
    // Dispose of effect resources
    echoBuffer = null;
}

```

DiscardQueuedFrames method

The [DiscardQueuedFrames](#) method is called when your effect should reset. A typical scenario for this is if your effect stores previously processed frames to use in processing the current frame. When this method is called, you should dispose of the set of previous frames you saved. This method can be used to reset any state related to previous frames, not only accumulated audio frames.

C#

```
public void DiscardQueuedFrames()
{
    // Reset contents of the samples buffer
    Array.Clear(echoBuffer, 0, echoBuffer.Length - 1);
    currentActiveSampleIndex = 0;
}
```

TimeIndependent property

The TimeIndependent [TimeIndependent](#) property lets the system know if your effect does not require uniform timing. When set to true, the system can use optimizations that enhance effect performance.

C#

```
public bool TimeIndependent { get { return true; } }
```

UseInputFrameForOutput property

Set the [UseInputFrameForOutput](#) property to **true** to tell the system that your effect will write its output to the audio buffer of the [InputFrame](#) of the [ProcessAudioFrameContext](#) passed into [ProcessFrame](#) instead of writing to the [OutputFrame](#).

C#

```
public bool UseInputFrameForOutput { get { return false; } }
```

Adding your custom effect to your app

To use your audio effect from your app, you must add a reference to the effect project to your app.

1. In Solution Explorer, under your app project, right-click **References** and select **Add reference**.
2. Expand the **Projects** tab, select **Solution**, and then select the check box for your effect project name. For this example, the name is *AudioEffectComponent*.
3. Click **OK**

If your audio effect class is declared in a different namespace, be sure to include that namespace in your code file.

C#

```
using AudioEffectComponent;
```

Add your custom effect to an AudioGraph node

For general information about using audio graphs, see [Audio graphs](#). The following code snippet shows you how to add the example echo effect shown in this article to an audio graph node. First, a [PropertySet](#) is created and a value for the **Mix** property, defined by the effect, is set. Next, the [AudioEffectDefinition](#) constructor is called, passing in the full class name of the custom effect type and the property set. Finally, the effect definition is added to the [EffectDefinitions](#) property of an existing [FileInputNode](#), causing the audio emitted to be processed by the custom effect.

C#

```
// Create a property set and add a property/value pair
PropertySet echoProperties = new PropertySet();
echoProperties.Add("Mix", 0.5f);

// Instantiate the custom effect defined in the 'AudioEffectComponent'
// project
AudioEffectDefinition echoEffectDefinition = new
AudioEffectDefinition(typeof(ExampleAudioEffect).FullName, echoProperties);
fileInputNode.EffectDefinitions.Add(echoEffectDefinition);
```

After it has been added to a node, the custom effect can be disabled by calling [DisableEffectsByDefinition](#) and passing in the [AudioEffectDefinition](#) object. For more information about using audio graphs in your app, see [AudioGraph](#).

Add your custom effect to a clip in a MediaComposition

The following code snippet demonstrates adding the custom audio effect to a video clip and a background audio track in a media composition. For general guidance for creating media compositions from video clips and adding background audio tracks, see [Media compositions and editing](#).

C#

```
// Create a property set and add a property/value pair
PropertySet echoProperties = new PropertySet();
echoProperties.Add("Mix", 0.5f);

// Instantiate the custom effect defined in the 'AudioEffectComponent'
project
AudioEffectDefinition echoEffectDefinition = new
AudioEffectDefinition(typeof(ExampleAudioEffect).FullName, echoProperties);

// Add custom audio effect to the current clip in the timeline
var currentClip = composition.Clips.FirstOrDefault(
    mc => mc.StartTimeInComposition <=
mediaPlayerElement.MediaPlayer.PlaybackSession.Position &&
    mc.EndTimeInComposition >=
mediaPlayerElement.MediaPlayer.PlaybackSession.Position);
currentClip.AudioEffectDefinitions.Add(echoEffectDefinition);

// Add custom audio effect to the first background audio track
if (composition.BackgroundAudioTracks.Count > 0)
{

composition.BackgroundAudioTracks[0].AudioEffectDefinitions.Add(echoEffectDe
finition);
}
```

Related topics

- [Simple camera preview access](#)
- [Media compositions and editing](#)
- [Win2D documentation](#) 
- [Media playback](#)

Feedback

Was this page helpful?

 Yes

 No

Media compositions and editing

Article • 10/20/2022

This article shows you how to use the APIs in the [Windows.Media Editing](#) namespace to quickly develop apps that enable the users to create media compositions from audio and video source files. Features of the framework include the ability to programmatically append multiple video clips together, add video and image overlays, add background audio, and apply both audio and video effects. Once created, media compositions can be rendered into a flat media file for playback or sharing, but compositions can also be serialized to and deserialized from disk, allowing the user to load and modify compositions that they have previously created. All of this functionality is provided in an easy-to-use Windows Runtime interface that dramatically reduces the amount and complexity of code required to perform these tasks when compared to the low-level [Microsoft Media Foundation](#) API.

Create a new media composition

The [MediaComposition](#) class is the container for all of the media clips that make up the composition and is responsible for rendering the final composition, loading and saving compositions to disc, and providing a preview stream of the composition so that the user can view it in the UI. To use **MediaComposition** in your app, include the [Windows.Media Editing](#) namespace as well as the [Windows.Media.Core](#) namespace that provides related APIs that you will need.

C#

```
using Windows.Media.Editing;
using Windows.Media.Core;
using Windows.Media.Playback;
using System.Threading.Tasks;
```

The **MediaComposition** object will be accessed from multiple points in your code, so typically you will declare a member variable in which to store it.

C#

```
private MediaComposition composition;
```

The constructor for **MediaComposition** takes no arguments.

C#

```
composition = new MediaComposition();
```

Add media clips to a composition

Media compositions typically contain one or more video clips. You can use a [FileOpenPicker](#) to allow the user to select a video file. Once the file has been selected, create a new [MediaClip](#) object to contain the video clip by calling [MediaClip.CreateFromFileAsync](#). Then you add the clip to the [MediaComposition](#) object's [Clips](#) list.

C#

```
private async Task PickFileAndAddClip()
{
    var picker = new Windows.Storage.Pickers.FileOpenPicker();
    picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.VideosLibrary;
    picker.FileTypeFilter.Add(".mp4");
    Windows.Storage.StorageFile pickedFile = await
picker.PickSingleFileAsync();
    if (pickedFile == null)
    {
        ShowErrorMessage("File picking cancelled");
        return;
    }

    // These files could be picked from a location that we won't have access
to later
    var storageItemAccessList =
Windows.Storage.AccessCache.StorageApplicationPermissions.FutureAccessList;
    storageItemAccessList.Add(pickedFile);

    var clip = await MediaClip.CreateFromFileAsync(pickedFile);
    composition.Clips.Add(clip);
}
```

- Media clips appear in the **MediaComposition** in the same order as they appear in [Clips](#) list.
- A **MediaClip** can only be included in a composition once. Attempting to add a **MediaClip** that is already being used by the composition will result in an error. To reuse a video clip multiple times in a composition, call [Clone](#) to create new **MediaClip** objects which can then be added to the composition.

- Universal Windows apps do not have permission to access the entire file system. The [FutureAccessList](#) property of the [StorageApplicationPermissions](#) class allows your app to store a record of a file that has been selected by the user so that you can retain permissions to access the file. The [FutureAccessList](#) has a maximum of 1000 entries, so your app needs to manage the list to make sure it does not become full. This is especially important if you plan to support loading and modifying previously created compositions.
- A [MediaComposition](#) supports video clips in MP4 format.
- If a video file contains multiple embedded audio tracks, you can select which audio track is used in the composition by setting the [SelectedEmbeddedAudioTrackIndex](#) property.
- Create a [MediaClip](#) with a single color filling the entire frame by calling [CreateFromColor](#) and specifying a color and a duration for the clip.
- Create a [MediaClip](#) from an image file by calling [CreateFromImageFileAsync](#) and specifying an image file and a duration for the clip.
- Create a [MediaClip](#) from an [IDirect3DSurface](#) by calling [CreateFromSurface](#) and specifying a surface and a duration from the clip.

Preview the composition in a MediaElement

To enable the user to view the media composition, add a [MediaPlayerElement](#) to the XAML file that defines your UI.

XML

```
<MediaPlayerElement x:Name="mediaPlayerElement" AutoPlay="False" Margin="5"
HorizontalAlignment="Stretch" AreTransportControlsEnabled="True" />
```

Declare a member variable of type [MediaStreamSource](#).

C#

```
private MediaStreamSource mediaStreamSource;
```

Call the [MediaComposition](#) object's [GeneratePreviewMediaStreamSource](#) method to create a [MediaStreamSource](#) for the composition. Create a [MediaSource](#) object by calling the factory method [CreateFromMediaStreamSource](#) and assign it to the [Source](#) property of the [MediaPlayerElement](#). Now the composition can be viewed in the UI.

C#

```
public void UpdateMediaElementSource()
{
    mediaStreamSource = composition.GeneratePreviewMediaStreamSource(
        (int)mediaPlayerElement.ActualWidth,
        (int)mediaPlayerElement.ActualHeight);

    mediaPlayerElement.Source =
        MediaSource.CreateFromMediaStreamSource(mediaStreamSource);
}
```

- The **MediaComposition** must contain at least one media clip before calling [GeneratePreviewMediaStreamSource](#), or the returned object will be null.
- The **MediaElement** timeline is not automatically updated to reflect changes in the composition. It is recommended that you call both **GeneratePreviewMediaStreamSource** and set the **MediaPlayerElement Source** property every time you make a set of changes to the composition and want to update the UI.

It is recommended that you set the **MediaStreamSource** object and the [Source](#) property of the **MediaPlayerElement** to null when the user navigates away from the page in order to release associated resources.

C#

```
protected override void OnNavigatedFrom(NavigationEventArgs e)
{
    mediaPlayerElement.Source = null;
    mediaStreamSource = null;
    base.OnNavigatedFrom(e);
}
```

Render the composition to a video file

To render a media composition to a flat video file so that it can be shared and viewed on other devices, you will need to use APIs from the [Windows.Media.Transcoding](#) namespace. To update the UI on the progress of the async operation, you will also need APIs from the [Windows.UI.Core](#) namespace.

C#

```
using Windows.Media.Transcoding;
using Windows.UI.Core;
```

After allowing the user to select an output file with a [FileSavePicker](#), render the composition to the selected file by calling the **MediaComposition** object's [RenderToFileAsync](#). The rest of the code in the following example simply follows the pattern of handling an [AsyncOperationWithProgress](#).

C#

```
private async Task RenderCompositionToFile()
{
    var picker = new Windows.Storage.Pickers.FileSavePicker();
    picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.VideosLibrary;
    picker.FileTypeChoices.Add("MP4 files", new List<string>() { ".mp4" });
    picker.SuggestedFileName = "RenderedComposition.mp4";

    Windows.Storage.StorageFile file = await picker.PickSaveFileAsync();
    if (file != null)
    {
        // Call RenderToFileAsync
        var saveOperation = composition.RenderToFileAsync(file,
MediaTrimmingPreference.Precise);

        saveOperation.Progress = new
AsyncOperationProgressHandler<TranscodeFailureReason, double>(async (info,
progress) =>
        {
            await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal,
new DispatchedHandler(() =>
            {
                ShowErrorMessage(string.Format("Saving file... Progress:
{0:F0}%", progress));
            }));
        });
        saveOperation.Completed = new
AsyncOperationWithProgressCompletedHandler<TranscodeFailureReason, double>
(async (info, status) =>
        {
            await this.Dispatcher.RunAsync(CoreDispatcherPriority.Normal,
new DispatchedHandler(() =>
            {
                try
                {
                    var results = info.GetResults();
                    if (results != TranscodeFailureReason.None || status !=
AsyncStatus.Completed)
                    {
                        ShowErrorMessage("Saving was unsuccessful");
                    }
                }
            }));
        });
    }
}
```

```

        else
        {
            ShowErrorMessage("Trimmed clip saved to file");
        }
    }
    finally
    {
        // Update UI whether the operation succeeded or not
    }

    }));
});
}
else
{
    ShowErrorMessage("User cancelled the file selection");
}
}

```

- The [MediaTrimmingPreference](#) allows you to prioritize speed of the transcoding operation versus the precision of trimming of adjacent media clips. **Fast** causes transcoding to be faster with lower-precision trimming, **Precise** causes transcoding to be slower but with more precise trimming.

Trim a video clip

Trim the duration of a video clip in a composition by setting the [MediaClip](#) objects [TrimTimeFromStart](#) property, the [TrimTimeFromEnd](#) property, or both.

C#

```

private void TrimClipBeforeCurrentPosition()
{
    var currentClip = composition.Clips.FirstOrDefault(
        mc => mc.StartTimeInComposition <=
mediaPlayerElement.MediaPlayer.PlaybackSession.Position &&
        mc.EndTimeInComposition >=
mediaPlayerElement.MediaPlayer.PlaybackSession.Position);

    TimeSpan positionFromStart =
mediaPlayerElement.MediaPlayer.PlaybackSession.Position -
currentClip.StartTimeInComposition;
    currentClip.TrimTimeFromStart = positionFromStart;
}

```

- You can use any UI that you want to let the user specify the start and end trim values. The example above uses the [Position](#) property of the

MediaPlaybackSession associated with the **MediaPlayerElement** to first determine which **MediaClip** is playing back at the current position in the composition by checking the **StartTimeInComposition** and **EndTimeInComposition**. Then the **Position** and **StartTimeInComposition** properties are used again to calculate the amount of time to trim from the beginning of the clip. The **FirstOrDefault** method is an extension method from the **System.Linq** namespace that simplifies the code for selecting items from a list.

- The **OriginalDuration** property of the **MediaClip** object lets you know the duration of the media clip without any clipping applied.
- The **TrimmedDuration** property lets you know the duration of the media clip after trimming is applied.
- Specifying a trimming value that is larger than the original duration of the clip does not throw an error. However, if a composition contains only a single clip and that is trimmed to zero length by specifying a large trimming value, a subsequent call to **GeneratePreviewMediaStreamSource** will return null, as if the composition has no clips.

Add a background audio track to a composition

To add a background track to a composition, load an audio file and then create a **BackgroundAudioTrack** object by calling the factory method **BackgroundAudioTrack.CreateFromFileAsync**. Then, add the **BackgroundAudioTrack** to the composition's **BackgroundAudioTracks** property.

C#

```
private async Task AddBackgroundAudioTrack()
{
    // Add background audio
    var picker = new Windows.Storage.Pickers.FileOpenPicker();
    picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.MusicLibrary;
    picker.FileTypeFilter.Add(".mp3");
    picker.FileTypeFilter.Add(".wav");
    picker.FileTypeFilter.Add(".flac");
    Windows.Storage.StorageFile audioFile = await
picker.PickSingleFileAsync();
    if (audioFile == null)
    {
        ShowErrorMessage("File picking cancelled");
        return;
    }

    // These files could be picked from a location that we won't have access
```

```

to later
    var storageItemAccessList =
        Windows.Storage.AccessCache.StorageApplicationPermissions.FutureAccessList;
    storageItemAccessList.Add(audioFile);

    var backgroundTrack = await
        BackgroundAudioTrack.CreateFromFileAsync(audioFile);

    composition.BackgroundAudioTracks.Add(backgroundTrack);
}

```

- A **MediaComposition** supports background audio tracks in the following formats: MP3, WAV, FLAC
- A background audio track
- As with video files, you should use the **StorageApplicationPermissions** class to preserve access to files in the composition.
- As with **MediaClip**, a **BackgroundAudioTrack** can only be included in a composition once. Attempting to add a **BackgroundAudioTrack** that is already being used by the composition will result in an error. To reuse an audio track multiple times in a composition, call **Clone** to create new **MediaClip** objects which can then be added to the composition.
- By default, background audio tracks begin playing at the start of the composition. If multiple background tracks are present, all of the tracks will begin playing at the start of the composition. To cause a background audio track to begin playback at another time, set the **Delay** property to the desired time offset.

Add an overlay to a composition

Overlays allow you to stack multiple layers of video on top of each other in a composition. A composition can contain multiple overlay layers, each of which can include multiple overlays. Create a **MediaOverlay** object by passing a **MediaClip** into its constructor. Set the position and opacity of the overlay, then create a new **MediaOverlayLayer** and add the **MediaOverlay** to its **Overlays** list. Finally, add the **MediaOverlayLayer** to the composition's **OverlayLayers** list.

C#

```

private void AddOverlay(MediaClip overlayMediaClip, double scale, double
left, double top, double opacity)
{
    Windows.Media.MediaProperties.VideoEncodingProperties encodingProperties

```

```

=
    overlayMediaClip.GetVideoEncodingProperties();

    Rect overlayPosition;

    overlayPosition.Width = (double)encodingProperties.Width * scale;
    overlayPosition.Height = (double)encodingProperties.Height * scale;
    overlayPosition.X = left;
    overlayPosition.Y = top;

    MediaOverlay mediaOverlay = new MediaOverlay(overlayMediaClip);
    mediaOverlay.Position = overlayPosition;
    mediaOverlay.Opacity = opacity;

    MediaOverlayLayer mediaOverlayLayer = new MediaOverlayLayer();
    mediaOverlayLayer.Overlays.Add(mediaOverlay);

    composition.OverlayLayers.Add(mediaOverlayLayer);
}

```

- Overlays within a layer are z-ordered based on their order in their containing layer's **Overlays** list. Higher indices within the list are rendered on top of lower indices. The same is true of overlay layers within a composition. A layer with higher index in the composition's **OverlayLayers** list will be rendered on top of lower indices.
- Because overlays are stacked on top of each other instead of being played sequentially, all overlays start playback at the beginning of the composition by default. To cause an overlay to begin playback at another time, set the **Delay** property to the desired time offset.

Add effects to a media clip

Each **MediaClip** in a composition has a list of audio and video effects to which multiple effects can be added. The effects must implement **IAudioEffectDefinition** and **IVideoEffectDefinition** respectively. The following example uses the current **MediaPlayerElement** position to choose the currently viewed **MediaClip** and then creates a new instance of the **VideoStabilizationEffectDefinition** and appends it to the media clip's **VideoEffectDefinitions** list.

C#

```

private void AddVideoEffect()
{
    var currentClip = composition.Clips.FirstOrDefault(
        mc => mc.StartTimeInComposition <=
        mediaPlayerElement.MediaPlayer.PlaybackSession.Position &&

```

```

        mc.EndTimeInComposition >=
mediaPlayerElement.MediaPlayer.PlaybackSession.Position);

        VideoStabilizationEffectDefinition videoEffect = new
VideoStabilizationEffectDefinition();
        currentClip.VideoEffectDefinitions.Add(videoEffect);
    }

```

Save a composition to a file

Media compositions can be serialized to a file to be modified at a later time. Pick an output file and then call the [MediaComposition](#) method [SaveAsync](#) to save the composition.

```

C#

private async Task SaveComposition()
{
    var picker = new Windows.Storage.Pickers.FileSavePicker();
    picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.VideosLibrary;
    picker.FileTypeChoices.Add("Composition files", new List<string>() {
".cmp" });
    picker.SuggestedFileName = "SavedComposition";

    Windows.Storage.StorageFile compositionFile = await
picker.PickSaveFileAsync();
    if (compositionFile == null)
    {
        ShowErrorMessage("User cancelled the file selection");
    }
    else
    {
        var action = composition.SaveAsync(compositionFile);
        action.Completed = (info, status) =>
        {
            if (status != AsyncStatus.Completed)
            {
                ShowErrorMessage("Error saving composition");
            }
        };
    }
}

```

Load a composition from a file

Media compositions can be deserialized from a file to allow the user to view and modify the composition. Pick a composition file and then call the [MediaComposition](#) method [LoadAsync](#) to load the composition.

C#

```
private async Task OpenComposition()
{
    var picker = new Windows.Storage.Pickers.FileOpenPicker();
    picker.SuggestedStartLocation =
Windows.Storage.Pickers.PickerLocationId.VideosLibrary;
    picker.FileTypeFilter.Add(".cmp");

    Windows.Storage.StorageFile compositionFile = await
picker.PickSingleFileAsync();
    if (compositionFile == null)
    {
        ShowErrorMessage("File picking cancelled");
    }
    else
    {
        composition = null;
        composition = await MediaComposition.LoadAsync(compositionFile);

        if (composition != null)
        {
            UpdateMediaElementSource();
        }
        else
        {
            ShowErrorMessage("Unable to open composition");
        }
    }
}
```

- If a media file in the composition is not in a location that can be accessed by your app and is not in the [FutureAccessList](#) property of the [StorageApplicationPermissions](#) class for your app, an error will be thrown when loading the composition.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Audio device information properties

Article • 10/20/2022

This article lists the device information properties related to audio devices. On Windows, each hardware device has associated [DeviceInformation](#) properties providing detailed information about a device that you can use when you need specific information about the device or when you are building a device selector. For general information about enumerating devices on Windows, see [Enumerate devices](#) and [Device information properties](#).

Name	Type	Description
System.Devices.AudioDevice.Microphone.SensitivityInDbfs	Double	Specifies the microphone sensitivity in decibels relative to full scale (dBFS) units.
System.Devices.AudioDevice.Microphone.SignalToNoiseRatioInDb	Double	Specifies the microphone signal to noise ratio (SNR) measured in decibel (dB) units.
System.Devices.AudioDevice.SpeechProcessingSupported	Boolean	Indicates whether the audio device supports speech processing.
System.Devices.AudioDevice.RawProcessingSupported	Boolean	Indicates whether the audio device supports raw processing.
System.Devices.MicrophoneArray.Geometry	unsigned char[]	Geometry data for a microphone array.

Related topics

- [Enumerate devices](#)
- [Device information properties](#)
- [Build a device selector](#)
- [Media playback](#)

Detect and respond to audio state changes

Article • 10/20/2022

Starting with Windows 10, version 1803, your app can detect when the system lowers or mutes the audio level of an audio stream your app is using. You can receive notifications for capture and render streams, for a particular audio device and audio category, or for a [MediaPlayer](#) object your app is using for media playback. For example, the system may lower, or "duck", the audio playback level when an alarm is ringing. The system will mute your app when it goes into the background if your app has not declared the *backgroundMediaPlayback* capability in the app manifest.

The pattern for handling audio state changes is the same for all supported audio streams. First, create an instance of the [AudioStateMonitor](#) class. In the following example, the app is using the [MediaCapture](#) class to capture audio for game chat. A factory method is called to get an audio state monitor associated with the game chat audio capture stream of the default communications device. Next, a handler is registered for the [SoundLevelChanged](#) event, which will be raised when the audio level for the associated stream is changed by the system.

C#

```
AudioStateMonitor gameChatAudioStateMonitor;
```

C#

```
string deviceId =  
Windows.Media.Devices.MediaDevice.GetDefaultAudioCaptureId(Windows.Media.Dev  
ices.AudioDeviceRole.Communications);  
gameChatAudioStateMonitor =  
AudioStateMonitor.CreateForCaptureMonitoringWithCategoryAndDeviceId(MediaCat  
egory.GameChat, deviceId);  
gameChatAudioStateMonitor.SoundLevelChanged += GameChatSoundLevelChanged;
```

In the [SoundLevelChanged](#) event handler, check the [SoundLevel](#) property of the [AudioStateMonitor](#) sender passed into the handler to determine what the new audio level for the stream is. In this example, the app stops capturing audio when the sound level is muted and resumes capturing when the audio level returns to full volume.

C#

```

private void GameChatSoundLevelChanged(AudioStateMonitor sender, object
args)
{
    switch (sender.SoundLevel)
    {
        case SoundLevel.Full:
            StartAudioCapture();
            break;
        case SoundLevel.Muted:
            StopAudioCapture();
            break;
        case SoundLevel.Low:
            // Audio capture should never be "ducked", only muted or full
            volume.
            Debug.WriteLine("Unexpected audio state change.");
            break;
    }
}

```

For more information on capturing audio with **MediaCapture**, see [Basic photo, video, and audio capture with MediaCapture](#).

Every instance of the **MediaPlayer** class has an **AudioStateMonitor** associated with it that you can use to detect when the system changes the volume level of the content currently being played. You may decide to handle audio state changes differently depending on what type of content is being played. For example, you may decide to pause playback of a podcast when the audio is lowered, but continue playback if the content is music.

C#

```

bool isPodcast;
bool isPausedDueToAudioStateMonitor;

```

C#

```

private void
AudioStateMonitor_SoundLevelChanged(Windows.Media.Audio.AudioStateMonitor
sender, object args)
{
    if ((sender.SoundLevel == SoundLevel.Full) || (sender.SoundLevel ==
SoundLevel.Low && !isPodcast))
    {
        if (isPausedDueToAudioStateMonitor)
        {
            mediaPlayer.Play();
            isPausedDueToAudioStateMonitor = false;
        }
    }
}

```

```
    else if ((sender.SoundLevel == SoundLevel.Muted) ||  
             (sender.SoundLevel == SoundLevel.Low && isPodcast))  
    {  
        if (mediaPlayer.PlaybackSession.PlaybackState ==  
MediaPlaybackState.Playing)  
        {  
            mediaPlayer.Pause();  
            isPausedDueToAudioStateMonitor = true;  
        }  
    }  
  
}
```

For more information on using **MediaPlayer**, see [Play audio and video with MediaPlayer](#).

Related topics

Create, edit, and save bitmap images

Article • 10/20/2022

This article explains how to load and save image files using [BitmapDecoder](#) and [BitmapEncoder](#) and how to use the [SoftwareBitmap](#) object to represent bitmap images.

The **SoftwareBitmap** class is a versatile API that can be created from multiple sources including image files, [WritableBitmap](#) objects, Direct3D surfaces, and code.

SoftwareBitmap allows you to easily convert between different pixel formats and alpha modes, and allows low-level access to pixel data. Also, **SoftwareBitmap** is a common interface used by multiple features of Windows, including:

- [CapturedFrame](#) allows you to get frames captured by the camera as a **SoftwareBitmap**.
- [VideoFrame](#) allows you to get a **SoftwareBitmap** representation of a **VideoFrame**.
- [FaceDetector](#) allows you to detect faces in a **SoftwareBitmap**.

The sample code in this article uses APIs from the following namespaces.

C#

```
using Windows.Storage;
using Windows.Storage.Pickers;
using Windows.Storage.Streams;
using Windows.Graphics.Imaging;
using Windows.UI.Xaml.Media.Imaging;
```

Create a SoftwareBitmap from an image file with BitmapDecoder

To create a **SoftwareBitmap** from a file, get an instance of [StorageFile](#) containing the image data. This example uses a [FileOpenPicker](#) to allow the user to select an image file.

C#

```
FileOpenPicker fileOpenPicker = new FileOpenPicker();
fileOpenPicker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;
fileOpenPicker.FileTypeFilter.Add(".jpg");
fileOpenPicker.ViewMode = PickerViewMode.Thumbnail;

var inputFile = await fileOpenPicker.PickSingleFileAsync();
```

```

if (inputFile == null)
{
    // The user cancelled the picking operation
    return;
}

```

Call the **OpenAsync** method of the **StorageFile** object to get a random access stream containing the image data. Call the static method **BitmapDecoder.CreateAsync** to get an instance of the **BitmapDecoder** class for the specified stream. Call **GetSoftwareBitmapAsync** to get a **SoftwareBitmap** object containing the image.

```

C#

SoftwareBitmap softwareBitmap;

using (IRandomAccessStream stream = await
inputFile.OpenAsync(FileAccessMode.Read))
{
    // Create the decoder from the stream
    BitmapDecoder decoder = await BitmapDecoder.CreateAsync(stream);

    // Get the SoftwareBitmap representation of the file
    softwareBitmap = await decoder.GetSoftwareBitmapAsync();
}

```

Save a SoftwareBitmap to a file with BitmapEncoder

To save a **SoftwareBitmap** to a file, get an instance of **StorageFile** to which the image will be saved. This example uses a **FileSavePicker** to allow the user to select an output file.

```

C#

FileSavePicker fileSavePicker = new FileSavePicker();
fileSavePicker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;
fileSavePicker.FileTypeChoices.Add("JPEG files", new List<string>() { ".jpg"
});
fileSavePicker.SuggestedFileName = "image";

var outputFile = await fileSavePicker.PickSaveFileAsync();

if (outputFile == null)
{
    // The user cancelled the picking operation
}

```

```
        return;  
    }
```

Call the [OpenAsync](#) method of the **StorageFile** object to get a random access stream to which the image will be written. Call the static method [BitmapEncoder.CreateAsync](#) to get an instance of the **BitmapEncoder** class for the specified stream. The first parameter to **CreateAsync** is a GUID representing the codec that should be used to encode the image. **BitmapEncoder** class exposes a property containing the ID for each codec supported by the encoder, such as [JpegEncoderId](#).

Use the [SetSoftwareBitmap](#) method to set the image that will be encoded. You can set values of the [BitmapTransform](#) property to apply basic transforms to the image while it is being encoded. The [IsThumbnailGenerated](#) property determines whether a thumbnail is generated by the encoder. Note that not all file formats support thumbnails, so if you use this feature, you should catch the unsupported operation error that will be thrown if thumbnails are not supported.

Call [FlushAsync](#) to cause the encoder to write the image data to the specified file.

C#

```
private async void SaveSoftwareBitmapToFile(SoftwareBitmap softwareBitmap,  
StorageFile outputFile)  
{  
    using (IRandomAccessStream stream = await  
outputFile.OpenAsync(FileAccessMode.ReadWrite))  
    {  
        // Create an encoder with the desired format  
        BitmapEncoder encoder = await  
BitmapEncoder.CreateAsync(BitmapEncoder.JpegEncoderId, stream);  
  
        // Set the software bitmap  
        encoder.SetSoftwareBitmap(softwareBitmap);  
  
        // Set additional encoding parameters, if needed  
        encoder.BitmapTransform.ScaledWidth = 320;  
        encoder.BitmapTransform.ScaledHeight = 240;  
        encoder.BitmapTransform.Rotation =  
Windows.Graphics.Imaging.BitmapRotation.Clockwise90Degrees;  
        encoder.BitmapTransform.InterpolationMode =  
BitmapInterpolationMode.Fant;  
        encoder.IsThumbnailGenerated = true;  
  
        try  
        {  
            await encoder.FlushAsync();  
        }  
        catch (Exception err)  
        {  

```

```

        const int WINCODEC_ERR_UNSUPPORTEDOPERATION =
unchecked((int)0x88982F81);
        switch (err.HResult)
        {
            case WINCODEC_ERR_UNSUPPORTEDOPERATION:
                // If the encoder does not support writing a thumbnail,
then try again
                // but disable thumbnail generation.
                encoder.IsThumbnailGenerated = false;
                break;
            default:
                throw;
        }
    }

    if (encoder.IsThumbnailGenerated == false)
    {
        await encoder.FlushAsync();
    }
}
}

```

You can specify additional encoding options when you create the **BitmapEncoder** by creating a new **BitmapPropertySet** object and populating it with one or more **BitmapTypedValue** objects representing the encoder settings. For a list of supported encoder options, see [BitmapEncoder options reference](#).

C#

```

var propertySet = new Windows.Graphics.Imaging.BitmapPropertySet();
var qualityValue = new Windows.Graphics.Imaging.BitmapTypedValue(
    1.0, // Maximum quality
    Windows.Foundation.PropertyType.Single
);

propertySet.Add("ImageQuality", qualityValue);

await Windows.Graphics.Imaging.BitmapEncoder.CreateAsync(
    Windows.Graphics.Imaging.BitmapEncoder.JpegEncoderId,
    stream,
    propertySet
);

```

Use SoftwareBitmap with a XAML Image control

To display an image within a XAML page using the **Image** control, first define an **Image** control in your XAML page.

XML

```
<Image x:Name="imageControl"/>
```

Currently, the **Image** control only supports images that use BGRA8 encoding and pre-multiplied or no alpha channel. Before attempting to display an image, test to make sure it has the correct format, and if not, use the **SoftwareBitmap** static **Convert** method to convert the image to the supported format.

Create a new **SoftwareBitmapSource** object. Set the contents of the source object by calling **SetBitmapAsync**, passing in a **SoftwareBitmap**. Then you can set the **Source** property of the **Image** control to the newly created **SoftwareBitmapSource**.

C#

```
if (softwareBitmap.BitmapPixelFormat != BitmapPixelFormat.Bgra8 ||
    softwareBitmap.BitmapAlphaMode == BitmapAlphaMode.Straight)
{
    softwareBitmap = SoftwareBitmap.Convert(softwareBitmap,
        BitmapPixelFormat.Bgra8, BitmapAlphaMode.Premultiplied);
}

var source = new SoftwareBitmapSource();
await source.SetBitmapAsync(softwareBitmap);

// Set the source of the Image control
imageControl.Source = source;
```

You can also use **SoftwareBitmapSource** to set a **SoftwareBitmap** as the **ImageSource** for an **ImageBrush**.

Create a SoftwareBitmap from a WriteableBitmap

You can create a **SoftwareBitmap** from an existing **WriteableBitmap** by calling **SoftwareBitmap.CreateCopyFromBuffer** and supplying the **PixelBuffer** property of the **WriteableBitmap** to set the pixel data. The second argument allows you to request a pixel format for the newly created **WriteableBitmap**. You can use the **PixelWidth** and **PixelHeight** properties of the **WriteableBitmap** to specify the dimensions of the new image.

C#

```
SoftwareBitmap outputBitmap = SoftwareBitmap.CreateCopyFromBuffer(  
    writeableBitmap.PixelBuffer,  
    BitmapPixelFormat.Bgra8,  
    writeableBitmap.PixelWidth,  
    writeableBitmap.PixelHeight  
);
```

Create or edit a SoftwareBitmap programmatically

So far this topic has addressed working with image files. You can also create a new **SoftwareBitmap** programmatically in code and use the same technique to access and modify the **SoftwareBitmap**'s pixel data.

SoftwareBitmap uses COM interop to expose the raw buffer containing the pixel data.

To use COM interop, you must include a reference to the **System.Runtime.InteropServices** namespace in your project.

C#

```
using System.Runtime.InteropServices;
```

Initialize the **IMemoryBufferByteAccess** COM interface by adding the following code within your namespace.

C#

```
[ComImport]  
[Guid("5B0D3235-4DBA-4D44-865E-8F1D0E4FD04D")]  
[InterfaceType(ComInterfaceType.InterfaceIsIUnknown)]  
unsafe interface IMemoryBufferByteAccess  
{  
    void GetBuffer(out byte* buffer, out uint capacity);  
}
```

Create a new **SoftwareBitmap** with pixel format and size you want. Or, use an existing **SoftwareBitmap** for which you want to edit the pixel data. Call **SoftwareBitmap.LockBuffer** to obtain an instance of the **BitmapBuffer** class representing the pixel data buffer. Cast the **BitmapBuffer** to the **IMemoryBufferByteAccess** COM interface and then call **IMemoryBufferByteAccess.GetBuffer** to populate a byte array with data. Use the

`BitmapBuffer.GetPlaneDescription` method to get a `BitmapPlaneDescription` object that will help you calculate the offset into the buffer for each pixel.

C#

```
softwareBitmap = new SoftwareBitmap(BitmapPixelFormat.Bgra8, 100, 100,
BitmapAlphaMode.Premultiplied);

using (BitmapBuffer buffer =
softwareBitmap.LockBuffer(BitmapBufferAccessMode.Write))
{
    using (var reference = buffer.CreateReference())
    {
        byte* dataInBytes;
        uint capacity;
        ((IMemoryBufferByteAccess)reference).GetBuffer(out dataInBytes, out
capacity);

        // Fill-in the BGRA plane
        BitmapPlaneDescription bufferLayout = buffer.GetPlaneDescription(0);
        for (int i = 0; i < bufferLayout.Height; i++)
        {
            for (int j = 0; j < bufferLayout.Width; j++)
            {
                byte value = (byte)((float)j / bufferLayout.Width * 255);
                dataInBytes[bufferLayout.StartIndex + bufferLayout.Stride *
i + 4 * j + 0] = value;
                dataInBytes[bufferLayout.StartIndex + bufferLayout.Stride *
i + 4 * j + 1] = value;
                dataInBytes[bufferLayout.StartIndex + bufferLayout.Stride *
i + 4 * j + 2] = value;
                dataInBytes[bufferLayout.StartIndex + bufferLayout.Stride *
i + 4 * j + 3] = (byte)255;
            }
        }
    }
}
```

Because this method accesses the raw buffer underlying the Windows Runtime types, it must be declared using the **unsafe** keyword. You must also configure your project in Microsoft Visual Studio to allow the compilation of unsafe code by opening the project's **Properties** page, clicking the **Build** property page, and selecting the **Allow Unsafe Code** checkbox.

Create a SoftwareBitmap from a Direct3D surface

To create a **SoftwareBitmap** object from a Direct3D surface, you must include the [Windows.Graphics.DirectX.Direct3D11](#) namespace in your project.

C#

```
using Windows.Graphics.DirectX.Direct3D11;
```

Call [CreateCopyFromSurfaceAsync](#) to create a new **SoftwareBitmap** from the surface. As the name indicates, the new **SoftwareBitmap** has a separate copy of the image data. Modifications to the **SoftwareBitmap** will not have any effect on the Direct3D surface.

C#

```
private async void CreateSoftwareBitmapFromSurface(IDirect3DSurface surface)
{
    softwareBitmap = await
    SoftwareBitmap.CreateCopyFromSurfaceAsync(surface);
}
```

Convert a SoftwareBitmap to a different pixel format

The **SoftwareBitmap** class provides the static method, [Convert](#), that allows you to easily create a new **SoftwareBitmap** that uses the pixel format and alpha mode you specify from an existing **SoftwareBitmap**. Note that the newly created bitmap has a separate copy of the image data. Modifications to the new bitmap will not affect the source bitmap.

C#

```
SoftwareBitmap bitmapBgra8 = SoftwareBitmap.Convert(softwareBitmap,
BitmapPixelFormat.Bgra8, BitmapAlphaMode.Premultiplied);
```

Transcode an image file

You can transcode an image file directly from a [BitmapDecoder](#) to a [BitmapEncoder](#). Create a [IRandomAccessStream](#) from the file to be transcoded. Create a new **BitmapDecoder** from the input stream. Create a new [InMemoryRandomAccessStream](#) for the encoder to write to and call [BitmapEncoder.CreateForTranscodingAsync](#), passing in the in-memory stream and the decoder object. Encode options are not supported when transcoding; instead you should use [CreateAsync](#). Any properties in the

input image file that you do not specifically set on the encoder, will be written to the output file unchanged. Call [FlushAsync](#) to cause the encoder to encode to the in-memory stream. Finally, seek the file stream and the in-memory stream to the beginning and call [CopyAsync](#) to write the in-memory stream out to the file stream.

C#

```
private async void TranscodeImageFile(StorageFile imageFile)
{

    using (IRandomAccessStream fileStream = await
imageFile.OpenAsync(FileAccessMode.ReadWrite))
    {
        BitmapDecoder decoder = await BitmapDecoder.CreateAsync(fileStream);

        var memStream = new
Windows.Storage.Streams.InMemoryRandomAccessStream();
        BitmapEncoder encoder = await
BitmapEncoder.CreateForTranscodingAsync(memStream, decoder);

        encoder.BitmapTransform.ScaledWidth = 320;
        encoder.BitmapTransform.ScaledHeight = 240;

        await encoder.FlushAsync();

        memStream.Seek(0);
        fileStream.Seek(0);
        fileStream.Size = 0;
        await RandomAccessStream.CopyAsync(memStream, fileStream);

        memStream.Dispose();
    }
}
```

Related topics

- [BitmapEncoder options reference](#)
- [Image Metadata](#)

Feedback

Was this page helpful?

 Yes

 No

Process bitmaps with OpenCV

Article • 10/20/2022

This article explains how to use the [SoftwareBitmap](#) class, which is used by many different Windows Runtime APIs to represent images, with the Open Source Computer Vision Library (OpenCV), an open source, native code library that provides a wide variety of image processing algorithms.

The examples in this article walk you through creating a native code Windows Runtime component that can be used from a UWP app, including apps that are created using C#. This helper component will expose a single method, **Blur**, which will use OpenCV's blur image processing function. The component implements private methods that get a pointer to the underlying image data buffer which can be used directly by the OpenCV library, making it simple to extend the helper component to implement other OpenCV processing features.

- For an introduction to using **SoftwareBitmap**, see [Create, edit, and save bitmap images](#).
- To learn how to use the OpenCV library, go to <https://opencv.org>.
- To see how to use the OpenCV helper component shown in this article with **MediaFrameReader** to implement real-time image processing of frames from a camera, see [Use OpenCV with MediaFrameReader](#).
- For a complete code example that implements some different effects, see the [Camera Frames + OpenCV Sample](#) in the Windows Universal Samples GitHub repo.

⚠ Note

The technique used by the OpenCVHelper component, described in detail in this article, requires that the image data to be processed resides in CPU memory, not GPU memory. So, for APIs that allow you to request the memory location of images, such as the [MediaCapture](#) class, you should specify CPU memory.

Create a helper Windows Runtime component for OpenCV interop

1. Add a new native code Windows Runtime component project to your solution

1. Add a new project to your solution in Visual Studio by right-clicking your solution in Solution Explorer and selecting **Add->New Project**.
2. Under the **Visual C++** category, select **Windows Runtime Component (Universal Windows)**. For this example, name the project "OpenCVBridge" and click **OK**.
3. In the **New Windows Universal Project** dialog, select the target and minimum OS version for your app and click **OK**.
4. Right-click the autogenerated file Class1.cpp in Solution Explorer and select **Remove**, when the confirmation dialog pops up, choose **Delete**. Then delete the Class1.h header file.
5. Right-click the OpenCVBridge project icon and select **Add->Class....** In the **Add Class** dialog, input "OpenCVHelper" in the **Class Name** field and then click **OK**. Code will be added to the created class files in a later step.

2. Add the OpenCV NuGet packages to your component project

1. Right-click the OpenCVBridge project icon in Solution Explorer and select **Manage NuGet Packages...**
2. When the NuGet Package Manager dialog opens, select the **Browse** tab and type "OpenCV.Win" in the search box.
3. Select "OpenCV.Win.Core" and click **Install**. In the **Preview** dialog, click **OK**.
4. Use the same procedure to install the "OpenCV.Win.ImgProc" package.

ⓘ Note

OpenCV.Win.Core and OpenCV.Win.ImgProc are not regularly updated and do not pass the Store compliance checks, therefore these packages are intended for experimentation only.

3. Implement the OpenCVHelper class

Paste the following code into the OpenCVHelper.h header file. This code includes OpenCV header files for the *Core* and *ImgProc* packages we installed and declares three methods that will be shown in the following steps.

C++

```
#pragma once

// OpenCVHelper.h
#include <opencv2\core\core.hpp>
```

```
#include <opencv2\imgproc\imgproc.hpp>

namespace OpenCVBridge
{
    public ref class OpenCVHelper sealed
    {
    public:
        OpenCVHelper() {}

        void Blur(
            Windows::Graphics::Imaging::SoftwareBitmap^ input,
            Windows::Graphics::Imaging::SoftwareBitmap^ output);

    private:
        // helper functions for getting a cv::Mat from SoftwareBitmap
        bool TryConvert(Windows::Graphics::Imaging::SoftwareBitmap^ from,
            cv::Mat& convertedMat);
        bool
        GetPointerToPixelData(Windows::Graphics::Imaging::SoftwareBitmap^ bitmap,
            unsigned char** pPixelData, unsigned int* capacity);
    };
}
```

Delete the existing contents of the OpenCVHelper.cpp file and then add the following include directives.

C++

```
#include "pch.h"
#include "OpenCVHelper.h"
#include "MemoryBuffer.h"
```

After the include directives, add the following **using** directives.

C++

```
using namespace OpenCVBridge;
using namespace Platform;
using namespace Windows::Graphics::Imaging;
using namespace Windows::Foundation;
using namespace Microsoft::WRL;
using namespace cv;
```

Next, add the method **GetPointerToPixelData** to OpenCVHelper.cpp. This method takes a **SoftwareBitmap** and, through a series of conversions, gets a COM interface representation of the pixel data through which we can get a pointer to the underlying data buffer as a **char** array.

First a **BitmapBuffer** containing the pixel data is obtained by calling **LockBuffer**, requesting a read/write buffer so that the OpenCV library can modify that pixel data. **CreateReference** is called to get an **IMemoryBufferReference** object. Next, the **IMemoryBufferByteAccess** interface is cast as an **IInspectable**, the base interface of all Windows Runtime classes, and **QueryInterface** is called to get an **IMemoryBufferByteAccess** COM interface that will allow us to obtain the pixel data buffer as a **char** array. Finally, populate the **char** array by calling **IMemoryBufferByteAccess::GetBuffer**. If any of the conversion steps in this method fail, the method returns **false**, indicating that further processing can't continue.

C++

```
bool OpenCVHelper::GetPointerToPixelData(SoftwareBitmap^ bitmap, unsigned
char** pPixelData, unsigned int* capacity)
{
    BitmapBuffer^ bmpBuffer = bitmap-
>LockBuffer(BitmapBufferAccessMode::ReadWrite);
    IMemoryBufferReference^ reference = bmpBuffer->CreateReference();

    ComPtr<IMemoryBufferByteAccess> pBufferByteAccess;
    if ((reinterpret_cast<IInspectable*>(reference)-
>QueryInterface(IID_PPV_ARGS(&pBufferByteAccess))) != S_OK)
    {
        return false;
    }

    if (pBufferByteAccess->GetBuffer(pPixelData, capacity) != S_OK)
    {
        return false;
    }
    return true;
}
```

Next, add the method **TryConvert** method shown below. This method takes a **SoftwareBitmap** and attempts to convert it to a **Mat** object, which is the matrix object OpenCV uses to represent image data buffers. This method calls the **GetPointerToPixelData** method defined above to get a **char** array representation of the pixel data buffer. If this succeeds, the constructor for the **Mat** class is called, passing in the pixel width and height obtained from the source **SoftwareBitmap** object.

⚠ Note

This example specifies the **CV_8UC4** constant as the pixel format for the created **Mat** object. This means that the **SoftwareBitmap** passed into this method must

have a `BitmapPixelFormat` property value of `BGRA8` with premultiplied alpha, the equivalent of `CV_8UC4`, to work with this example.

A shallow copy of the created **Mat** object is returned from the method so that further processing operates on the same data pixel data buffer referenced by the **SoftwareBitmap** and not a copy of this buffer.

C++

```
bool OpenCVHelper::TryConvert(SoftwareBitmap^ from, Mat& convertedMat)
{
    unsigned char* pPixels = nullptr;
    unsigned int capacity = 0;
    if (!GetPointerToPixelData(from, &pPixels, &capacity))
    {
        return false;
    }

    Mat mat(from->PixelHeight,
            from->PixelWidth,
            CV_8UC4, // assume input SoftwareBitmap is BGRA8
            (void*)pPixels);

    // shallow copy because we want convertedMat.data = pPixels
    // don't use .copyTo or .clone
    convertedMat = mat;
    return true;
}
```

Finally, this example helper class implements a single image processing method, **Blur**, which simply uses the **TryConvert** method defined above to retrieve a **Mat** object representing the source bitmap and the target bitmap for the blur operation, and then calls the **blur** method from the OpenCV `ImgProc` library. The other parameter to **blur** specifies the size of the blur effect in the X and Y directions.

C++

```
void OpenCVHelper::Blur(SoftwareBitmap^ input, SoftwareBitmap^ output)
{
    Mat inputMat, outputMat;
    if (!(TryConvert(input, inputMat) && TryConvert(output, outputMat)))
    {
        return;
    }
    blur(inputMat, outputMat, cv::Size(15, 15));
}
```

A simple SoftwareBitmap OpenCV example using the helper component

Now that the OpenCVBridge component has been created, we can create a simple C# app that uses the OpenCV **blur** method to modify a **SoftwareBitmap**. To access the Windows Runtime component from your UWP app, you must first add a reference to the component. In Solution Explorer, right-click the **References** node under your UWP app project and select **Add reference....** In the Reference Manager dialog, select **Projects->Solution**. Check the box next to the OpenCVBridge project and then click **OK**.

The example code below allows the user to select an image file and then uses **BitmapDecoder** to create a **SoftwareBitmap** representation of the image. For more information on working with **SoftwareBitmap**, see [Create, edit, and save bitmap images](#).

As discussed previously in this article, the **OpenCVHelper** class requires that all provided **SoftwareBitmap** images be encoded using the BGRA8 pixel format with premultiplied alpha values, so if the image is not already in this format, the example code calls **Convert** to convert the image into the expected format.

Next, a **SoftwareBitmap** is created to be used as the target of the blur operation. The input image properties are used as arguments to the constructor to create a bitmap with matching format.

A new instance of **OpenCVHelper** is created, and the **Blur** method is called, passing in the source and target bitmaps. Finally, a **SoftwareBitmapSource** is created to assign the output image to a XAML **Image** control.

This sample code uses APIs from the following namespaces, in addition to the namespaces included by the default project template.

C#

```
using Windows.Graphics.Imaging;
using Windows.Storage;
using Windows.Storage.Pickers;
using Windows.Storage.Streams;
using Windows.UI.Xaml.Media.Imaging;
```

C#

```
FileOpenPicker fileOpenPicker = new FileOpenPicker();
fileOpenPicker.SuggestedStartLocation = PickerLocationId.PicturesLibrary;
fileOpenPicker.FileTypeFilter.Add(".jpg");
fileOpenPicker.ViewMode = PickerViewMode.Thumbnail;
```

```

var inputFile = await fileOpenPicker.PickSingleFileAsync();

if (inputFile == null)
{
    // The user cancelled the picking operation
    return;
}

SoftwareBitmap inputBitmap;
using (IRandomAccessStream stream = await
inputFile.OpenAsync(FileAccessMode.Read))
{
    // Create the decoder from the stream
    BitmapDecoder decoder = await BitmapDecoder.CreateAsync(stream);

    // Get the SoftwareBitmap representation of the file
    inputBitmap = await decoder.GetSoftwareBitmapAsync();
}

if (inputBitmap.BitmapPixelFormat != BitmapPixelFormat.Bgra8
    || inputBitmap.BitmapAlphaMode != BitmapAlphaMode.Premultiplied)
{
    inputBitmap = SoftwareBitmap.Convert(inputBitmap,
    BitmapPixelFormat.Bgra8, BitmapAlphaMode.Premultiplied);
}

SoftwareBitmap outputBitmap = new
SoftwareBitmap(inputBitmap.BitmapPixelFormat, inputBitmap.PixelWidth,
inputBitmap.PixelHeight, BitmapAlphaMode.Premultiplied);

var helper = new OpenCVBridge.OpenCVHelper();
helper.Blur(inputBitmap, outputBitmap);

var bitmapSource = new SoftwareBitmapSource();
await bitmapSource.SetBitmapAsync(outputBitmap);
imageControl.Source = bitmapSource;

```

Related topics

- [BitmapEncoder options reference](#)
- [Image Metadata](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Process bitmaps with the LowLightFusion API

Article • 10/20/2022

Low-light images are difficult to capture with good image quality, especially on mobile devices with fixed aperture and sensor size. To compensate for low-lighting, devices may increase exposure time or sensor gain, which can lead to motion blur and increased noise in images.

The [LowLightFusion class](#) improves the quality of low-light images by sampling pixel information from multiple frames in close temporal proximity, i.e., short burst images, to reduce noise and motion blur. This is useful capability to add to a photo editing app, for example.

This feature is also made available through the [AdvancedPhotoCapture class](#), which applies the Low Light Fusion algorithm to a sequence of images directly after the images are captured, if needed. See [Low-light photo capture](#) to learn how to implement this feature.

Prepare the images for processing

In this example, we'll demonstrate how to use the [LowLightFusion class](#), as well as the [FileOpenPicker](#) to allow a user to select multiple images to perform Low Light Fusion on.

First, we'll need to determine how many images (also known as frames) the algorithm accepts, and create a list to hold these frames.

C#

```
// Query the supported max number of input bitmap frames for Low Light Fusion
int maxFrames = LowLightFusion.MaxSupportedFrameCount;

// The bitmap frames to perform Low Light Fusion on.
var framelist = new List<SoftwareBitmap>(maxFrames);
```

Once we determine how many frames the Low Light Fusion algorithm accepts, we can use the [FileOpenPicker](#) to allow the user to choose which images should be used in the algorithm.

C#